

Come la Linkage Unit costruisce, persiste e aggiorna la Link Table

Panoramica architetture ad alto livello — progetto PRIMAT

1 Introduzione

PRIMAT è un sistema di *Privacy-Preserving Record Linkage* (PPRL): più **Data Owner** (DO) — ciascuno titolare di una propria base dati contenente informazioni identificative su persone — vogliono scoprire quali dei loro record si riferiscono alla stessa entità reale, senza mai condividere i dati in chiaro tra loro. A occuparsi di questo compito è la **Linkage Unit** (LU): un componente centrale che riceve dai Data Owner solo impronte offuscate dei record (*Bloom Filter*), calcola quali impronte si assomigliano abbastanza da rappresentare la stessa persona, e mantiene nel tempo una **Link Table**: la struttura che collega i record delle varie sorgenti allo stesso identificativo pseudonimizzato.

Questo documento descrive, ad alto livello e senza entrare nei dettagli implementativi o matematici, quattro aspetti:

1. come la LU costruisce la Link Table la prima volta;
2. come questa struttura viene resa persistente;
3. come funziona l'aggiornamento incrementale quando arrivano nuovi record;
4. come l'ambiente così prodotto si presta naturalmente a essere interrogato da un motore di query federata (es. Trino) che unisce le sorgenti in una vista globale pseudonimizzata.

2 Costruzione iniziale della Link Table

2.1 Il flusso concettuale

Quando la Linkage Unit viene avviata per la prima volta su un insieme di Data Owner, non esiste ancora alcuna Link Table: viene costruita da zero seguendo quattro fasi concettuali, applicate a tutte le impronte ricevute:

1. **Blocking**: invece di confrontare ogni record con tutti gli altri (costo quadratico), i record vengono raggruppati in "blocchi" tramite chiavi calcolate con tecniche di *Locality Sensitive Hashing* (LSH). Solo i record che condividono almeno una chiave di blocco vengono confrontati fra loro.
2. **Calcolo della similarità**: per ogni coppia di record candidata si calcola un punteggio di somiglianza tra le rispettive impronte.
3. **Classificazione**: le coppie con similarità sopra una soglia configurabile vengono marcate come "match" (stessa entità probabile).
4. **Clustering**: i match vengono aggregati in gruppi (cluster) di record che rappresentano la stessa persona, anche quando la catena di somiglianze attraversa più di due sorgenti.

Il risultato di questa pipeline è proprio la Link Table iniziale: un insieme di cluster, ciascuno identificato da un id univoco (lo pseudonimo globale), che raggruppa i record provenienti da una o più sorgenti.

2.2 Configurabilità

L'intero processo è guidato da un file di configurazione dichiarativo, non da codice: si specificano le sorgenti (Data Owner) coinvolte, la soglia di similarità, i parametri dell'algoritmo di blocking e, soprattutto, la **strategia di clustering** da applicare. Sono disponibili più strategie intercambiabili, ciascuna con i propri parametri di tuning, pensate per scenari diversi (maggiore tolleranza al rumore, maggiore velocità, garanzie diverse sulla qualità dei cluster). La scelta della strategia e dei suoi parametri avviene quindi semplicemente cambiando la configurazione, senza toccare la logica applicativa.

2.3 Supporto di tutti i contesti dirty/clean

Una sorgente può essere "**dirty**" (può contenere più record che si riferiscono alla stessa persona, cioè ha bisogno anche di una deduplica interna) oppure "**clean**" (già deduplicata al suo interno). La LU supporta in modo nativo tutte le combinazioni:

- **dirty–dirty**: nessuna sorgente è già deduplicata;
- **dirty–clean (mista)**: una o più sorgenti pulite convivono con sorgenti sporche;
- **clean–clean**: tutte le sorgenti sono già deduplicate, serve solo collegarle tra loro;
- **sorgente singola (deduplica)**: un solo Data Owner "dirty" vuole scoprire i duplicati interni alla propria base dati, senza alcun secondo Data Owner coinvolto.

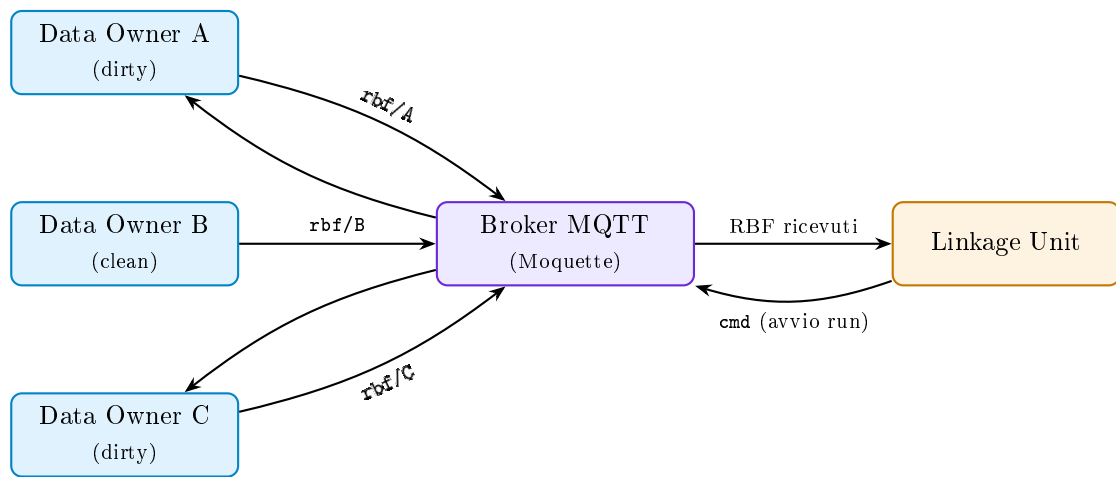
Ognuno di questi scenari è semplicemente una diversa combinazione di configurazione (quali sorgenti sono dichiarate "pulite" e quale strategia di clustering si sceglie), non un percorso di codice separato.

2.4 Multi-sorgente per natura

Il matching non è limitato a due sole sorgenti confrontate a coppie: la LU costruisce un unico grafo di similarità che può coinvolgere contemporaneamente un numero arbitrario di Data Owner. Un cluster può quindi raggruppare record provenienti da tre, quattro o più sorgenti differenti in un solo passaggio, se la catena di somiglianze lo giustifica.

2.5 Relazione tra Data Owner e Linkage Unit

I Data Owner non inviano mai dati identificativi in chiaro. Ogni Data Owner trasforma localmente i propri dati in un'impronta offuscata e irreversibile (un Bloom Filter costruito a partire da frammenti hashati dei campi identificativi) e la pubblica su un canale di messaggistica intermedio; la Linkage Unit si mette in ascolto su quello stesso canale, raccoglie le impronte di tutte le sorgenti coinvolte e avvia il calcolo. Il canale funge anche da tramite per i comandi di avvio inviati dalla LU ai Data Owner.



Sul canale viaggiano **solo impronte offuscate** (Bloom Filter) e digest di configurazione: mai valori identificativi in chiaro.

Figura 1: Relazione tra Data Owner e Linkage Unit: ciascun Data Owner pubblica la propria impronta offuscata su un broker di messaggistica condiviso; la Linkage Unit riceve le impronte di tutte le sorgenti coinvolte e coordina l'avvio di ogni esecuzione.

3 Persistenza della Link Table

3.1 Il modello di storage

La Link Table non è un file volatile ricalcolato da zero a ogni esecuzione: viene salvata in un database relazionale, così che le esecuzioni successive possano ripartire da quanto già consolidato. Concettualmente lo schema è composto da poche entità:

- **Party**: rappresenta un Data Owner registrato presso la LU.
- **Record**: l'impronta di un singolo record ricevuto da una Party, con un identificativo locale univoco (prefissato per evitare collisioni tra sorgenti diverse).
- **Cluster**: l'entità "reale" riconosciuta nel tempo — è di fatto la riga della Link Table: il suo id è lo pseudonimo globale condiviso da tutti i record che raggruppa.
- **Chiavi di blocco**: le stesse chiavi usate in fase di blocking vengono conservate accanto a record e cluster, così da poter individuare rapidamente, in futuro, quali cluster storici sono potenzialmente coinvolti da un nuovo record.

Ogni strategia di clustering scrive su un proprio spazio dati dedicato: questo mantiene isolate esecuzioni con strategie diverse e permette di confrontarne i risultati senza interferenze reciproche.

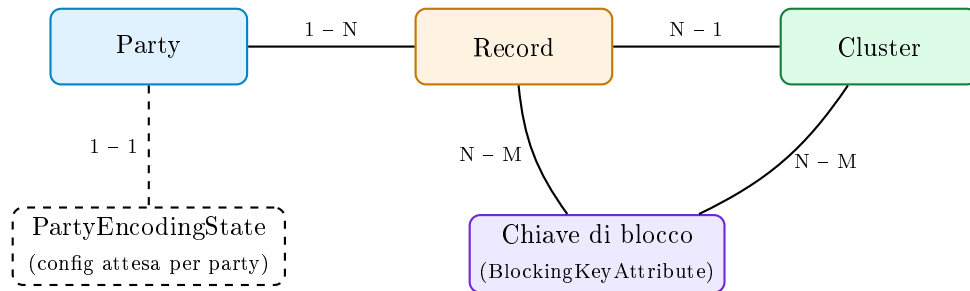


Figura 2: Schema concettuale della persistenza: ogni Party ha molti Record, ogni Record appartiene a un solo Cluster (finché non viene eventualmente fuso), Record e Cluster condividono le chiavi di blocco che permettono di ritrovarli rapidamente in futuro. **PartyEncodingState** verifica che una Party non cambi silenziosamente il modo in cui genera le proprie impronte tra un'esecuzione e l'altra.

3.2 Esempio minimo: prima inizializzazione

Supponiamo due Data Owner, **A** (dirty) e **B** (clean), che inviano alla LU le impronte di pochi record. Per rendere l'esempio leggibile, mostriamo qui i valori identificativi originali dei record: nella realtà questi valori *non lasciano mai* il Data Owner, che trasmette solo l'impronta offuscata corrispondente.

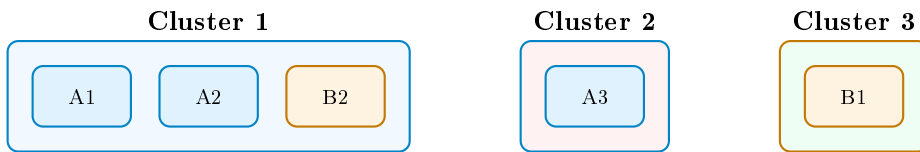
Tabella 1: Record in ingresso al primo run (valori mostrati solo a scopo illustrativo)

Id record	Party	Nome	Cognome
A1	A	Mario	Rossi
A2	A	Mario	Rossi
A3	A	Luca	Bianchi
B1	B	Maria	Verdi
B2	B	Mario	Rossi

Dopo blocking, calcolo di similarità, classificazione e clustering, la LU riconosce che **A1**, **A2** e **B2** rappresentano la stessa persona (Mario Rossi, con un duplicato interno alla sorgente A), mentre **A3** e **B1** restano singoli. La Link Table risultante:

Tabella 2: Link Table dopo la prima inizializzazione

Cluster (pseudonimo globale)	Record contenuti	Party coinvolte
Cluster 1	A1, A2, B2	A, B
Cluster 2	A3	A
Cluster 3	B1	B



L'id di ciascun cluster è lo pseudonimo globale usato per collegare record di sorgenti diverse senza mai esporre i dati originali.

Figura 3: Vista concettuale della Link Table: ogni cluster è un contenitore pseudonimizzato che raggruppa i record (di una o più sorgenti) riconosciuti come la stessa entità.

4 Logica di Incrementing

Una volta che la Link Table esiste, la LU non deve ripartire da zero ogni volta che arrivano nuovi record: li integra in modo incrementale.

4.1 Il principio

Quando arriva un nuovo lotto di record:

1. la LU non ricarica l'intero storico, ma recupera *solo* i cluster già esistenti che condividono almeno una chiave di blocco con uno dei record nuovi (i "candidati");
2. l'appartenenza ai cluster già persistiti viene presa come punto di partenza fisso: un legame stabilito in un'esecuzione precedente non può mai essere smontato da un'esecuzione successiva;
3. i nuovi match vengono poi applicati sopra questa base, e per ogni gruppo di record risultante si determina l'esito confrontandolo con i cluster storici coinvolti:
 - **nessun cluster storico coinvolto** → si crea un *nuovo cluster*;
 - **un solo cluster storico coinvolto** → il cluster *cresce*, incorporando i record nuovi;
 - **più cluster storici coinvolti** → i cluster vengono *fusi* in uno solo (l'id più basso viene mantenuto come identificativo del cluster risultante).

In altre parole: un cluster può solo crescere o fondersi con altri, mai essere diviso silenziosamente da un'esecuzione successiva.

4.2 Esempio minimo: arrivo di nuovi record

Riprendiamo la Link Table dell'esempio precedente (Cluster 1 = {A1, A2, B2}, Cluster 2 = {A3}, Cluster 3 = {B1}).

Caso 1 — crescita di un cluster. Un terzo Data Owner **C** invia un nuovo record **C1** = "Mario Rossi". La LU trova che **C1** corrisponde al Cluster 1 già esistente:

	Prima	Dopo l'arrivo di C1
Cluster 1	A1, A2, B2	A1, A2, B2, C1

Caso 2 — fusione di due cluster. Successivamente arriva un nuovo record **C2** da C che, a sua volta, risulta simile sia a A3 (Cluster 2) sia a B1 (Cluster 3): è un "record ponte" che rivela che i due cluster storici in realtà rappresentano la stessa persona.

	Prima	Dopo l'arrivo di C2
Cluster 2	A3	Cluster 2 (fuso): A3, B1, C2
Cluster 3	B1	

Il Cluster 3 viene svuotato e i suoi record vengono ripuntati sul Cluster 2 (id più basso), che diventa il cluster risultante della fusione.

5 Estensioni possibili: DELETE e UPDATE di record storici

Lo schema descritto finora copre l'inserimento di record nuovi, ma non ancora la loro rimozione o correzione. È ragionevole immaginare un'estensione in cui un Data Owner possa richiedere

esplicitamente, tramite un canale dedicato, la cancellazione o la modifica di un proprio record già linkato in passato. Si tratta di una funzionalità **non implementata a oggi**, qui discussa solo a livello di principio:

- **DELETE esplicito**: il Data Owner segnala l'id di un proprio record da rimuovere. La LU dovrebbe scollegare il record dal proprio cluster senza dover ripetere l'intero calcolo di linkage: se il cluster conteneva altri record, resta valido e si riduce soltanto; se il record era l'unico membro del cluster, il cluster stesso viene eliminato.
- **UPDATE esplicito**: una correzione ai dati di un record può essere trattata come una cancellazione seguita da un nuovo inserimento con l'impronta aggiornata, rientrando così nella normale logica di incrementing già descritta, senza bisogno di un percorso separato.
- **Implicazioni da considerare**: un canale di richiesta esplicito e autenticato lato Data Owner, tracciabilità delle modifiche per finalità di audit, e la possibilità che la rimozione di un record "ponte" renda necessario valutare se un cluster fuso in passato debba essere ricontrollato.

6 Verso l'interrogazione federata: come la Link Table prepara il terreno

L'integrazione di un motore di query federata (ad esempio Trino) che unisca le sorgenti dei Data Owner e la Link Table in un'unica vista globale è in corso di sviluppo separatamente e non è oggetto di questo documento. Vale però la pena osservare *perché* l'ambiente che la Linkage Unit già produce si presta naturalmente a questo tipo di interrogazione:

- **Schema relazionale standard**: la Link Table vive in un database relazionale interrogabile con un connettore JDBC comune, senza bisogno di adattatori dedicati o formati proprietari.
- **Chiave di join già pseudonimizzata**: l'id di cluster è esattamente lo pseudonimo globale che un motore federato userebbe come chiave di join tra le tabelle dei diversi Data Owner, senza mai dover transitare per dati identificativi in chiaro.
- **Identificativi senza collisioni**: gli id dei record sono prefissati per sorgente, così che una vista che unisce più Data Owner non incontri mai ambiguità tra record di provenienza diversa.
- **Spazi dati isolati e prevedibili**: ogni strategia di clustering scrive nel proprio spazio dati dedicato, così un motore di interrogazione può scegliere in modo esplicito quale "versione" della Link Table consultare, senza risultati misti o ambigui.
- **Coerenza già garantita a monte**: i controlli di coerenza che la LU esegue tra le sorgenti (stessa configurazione di codifica, stessa dimensione delle impronte) assicurano che i dati letti da un motore federato siano sempre internamente consistenti, senza bisogno che sia il motore di query a doverlo verificare.

In sintesi: il lavoro della Linkage Unit non produce soltanto i collegamenti tra i record, ma un ambiente dati già pulito, stabile e pseudonimizzato — esattamente il presupposto di cui un livello di virtual data integration ha bisogno per poter interrogare più sorgenti come se fossero un'unica vista coerente.